

Azure Storage 选型分析

业务场景专项分析报告

业务场景	ASP.NET MVC + 文件上传 + Windows Service 定期分类
分析目标	确定存储类型 + 数据库路径策略
核心问题	Blob vs File ? 数据库路径更新还是保留?
文档版本	V1.0 - 2026-04-15

目录

1. 业务场景描述
2. 当前架构分析
3. 存储类型选择: Blob vs File
4. 推荐方案: Azure File Storage
5. 数据库路径策略分析
6. 推荐方案: 路径映射层 (保留原路径)
7. 完整迁移架构设计
8. ASP.NET 代码改造方案
9. 迁移实施步骤
10. 总结与建议

1. 业务场景描述

1.1 业务流程概览

用户通过 ASP.NET Framework MVC 应用发起业务请求（Request），系统为每个 Request 生成唯一的 RequestID。用户上传 PDF 和 Office 文档，数据库记录文档与 RequestID 的关联关系。文档存储在本地文件系统，按 RequestID 分文件夹组织。此外，Windows Service 定时扫描这些文件夹，按 Branch（部门）和 SealType（印章类型）重新分类到新路径。

1.2 核心业务特点

- 多用户并发上传：多个用户可能同时上传文档到不同 RequestID
- 文件夹层级结构：RequestID -> Branch -> SealType 多级目录
- 定期自动分类：Windows Service 定时执行文件迁移和重分类
- 文档类型固定：主要是 PDF 和 Office 文档（Word、Excel）
- 数据库关联：文档路径与 RequestID 在数据库中强关联
- 业务连续性要求：迁移过程不能影响现有业务

1.3 当前文件组织结构

路径层级	示例路径	说明
根目录	D:\FileStorage\	文件存储根目录
一级目录	D:\FileStorage\[RequestID]\	按请求ID分组
二级目录	D:\FileStorage\[RequestID]\[Branch]\	按部门分类
三级目录	D:\FileStorage\[RequestID]\[Branch]\[SealType]\	按印章类型
文件	D:\FileStorage\[RequestID]\[Branch]\[SealType]\doc.pdf	实际文件

2. 当前架构分析

2.1 系统组件

组件	技术栈	功能描述
Web 应用	ASP.NET MVC	用户上传/下载文档，RequestID 管理
数据库	SQL Server	存储 Request-Document 关系、文件路径
本地存储	NTFS 文件系统	存储实际 PDF/Office 文件
Windows Service	.NET Windows Service	定时扫描、按 Branch/SealType 重分类

2.2 数据库表结构（推测）

表名	主要字段	说明
Request	RequestID, Branch, Status, CreatedAt	业务请求主表
Document	DocumentID, RequestID, FileName, FilePath, FileType, FileSize	文档表 (FilePath 存储本地路径)

2.3 关键依赖分析

当前系统对文件系统的依赖点：

- MVC Controller：使用 System.IO 读写文件、Directory.CreateDirectory 等
- Windows Service：使用 File.Move、Directory.GetFiles 扫描和移动文件
- 业务逻辑：根据 FilePath 字段拼接完整路径进行文件访问
- 前端展示：直接通过 URL 访问本地文件（可能需要映射虚拟目录）

3. 存储类型选择：Blob vs File

3.1 两种存储类型对比

对比项	Blob Storage	File Storage	本场景适合
数据结构	扁平 (Container/Blob)	层级 (Share/Directory/File)	File
访问协议	REST API	SMB 2.1+/3.0、NFS、REST API	File (SMB)
目录结构	虚拟目录 (命名约定)	真实目录层级	File
Windows 兼容	需 SDK 或 AzCopy	原生支持，像本地磁盘一样	File
文件移动/重分类	需下载+上传	支持 File.Move (部分场景)	File
定期扫描工具	Blob Inventory	AzCopy、Robocopy、PowerShell	File
Office/PDF 直读	需下载到本地	可挂载后直接打开	File
Web URL 访问	原生支持	需生成 SAS URL	Blob
数据库路径格式	https://xxx.blob.core.windows.net/	\\share\dir\file.ext	File
与现有代码兼容性	低 (需重写文件操作)	高 (使用 SMB 挂载)	File

3.2 本场景关键决策因素

决策因素	说明	权重
Windows Service 兼容性	现有 Service 使用 System.IO API, File 存储可直接兼容	高
文件夹层级结构	RequestID->Branch->SealType 多级目录, File 原生支持	高
定期扫描和移动	Service 需要遍历和移动文件, File 更适合	高
Office/PDF 直开	用户可能需要直接打开文档, File 可映射驱动器	中
Web URL 分享	偶尔需要分享链接, File 也支持 SAS URL	低
Blob 扁平结构	需要额外维护虚拟目录逻辑, 增加复杂度	低

4. 推荐方案：Azure File Storage

结论：推荐使用 Azure File Storage

原因：现有架构大量依赖传统文件系统 API，迁移到 File Storage 改动最小，

且完美支持现有的 RequestID->Branch->SealType 层级结构。

4.1 Azure File Storage 优势

- + SMB 协议原生支持：Windows Service 无需修改即可访问 Azure File Share
- + 目录结构完美匹配：Share 对应根目录，Directory 对应 RequestID/Branch/SealType
- + 最小代码改动：通过 SMB 挂载后，路径逻辑基本不变
- + 统一身份认证：可使用 Azure AD 集成或存储账户密钥
- + 高可用设计：LRS/ZRS/GRS 多重冗余，无需担心单点故障
- + 弹性扩展：单个 File Share 最大 100 TiB，按需扩展

4.2 迁移后的目录结构映射

本地路径	Azure File Storage 路径
D:\FileStorage\	\\mystorageaccount.file.core.windows.net\fileshare\
D:\FileStorage\[RequestID]\	fileshare/[RequestID]/
D:\FileStorage\[RequestID]\[Branch]\	fileshare/[RequestID]/[Branch]/
D:\FileStorage\[RequestID]\[Branch]\[SealType]\	fileshare/[RequestID]/[Branch]/[SealType]/

5. 数据库路径策略分析

5.1 两种策略对比

策略	策略A：更新为Azure路径	策略B：保留原路径（推荐）
做法	将 FilePath 更新为 Azure URL	保持 FilePath 不变，新增配置映射
优点	路径格式统一，数据一致性高	代码改动最小，支持回滚
缺点	需批量 UPDATE，影响大	需要维护映射逻辑
风险	迁移失败回滚复杂	配置错误可能找不到文件
适用场景	新项目或可接受大范围测试	生产环境、要求平滑迁移
本场景推荐	否	是

5.2 为什么推荐策略B（保留原路径）？

- 业务代码耦合深：Document.FilePath 在多个地方被引用，改动面太大
- 回滚风险高：如果迁移出问题，需要把路径再改回去
- Windows Service 依赖：Service 内部的路径拼接逻辑可能很复杂
- 测试成本：所有涉及路径的接口都需要重新测试
- 混合场景可能：未来可能需要同时支持本地和 Azure

重要提醒：数据库路径不需要更新！

建议保留数据库中的 FilePath 字段不变（存储相对路径），

通过配置层将本地根路径映射到 Azure File Share。

6. 推荐方案：路径映射层

6.1 架构设计

通过配置管理本地根路径和 Azure File Share 的映射关系。应用程序和 Windows Service 通过统一的文件服务获取完整路径，无需关心文件实际存储在哪里。

配置项	本地环境	Azure 环境
根路径配置	D:\FileStorage	\\xxx.file.core.windows.net\fileshare
数据库存储内容	\Request001\Branch1\SealType1\doc.pdf	\Request001\Branch1\SealType1\doc.pdf
代码获取的完整路径	D:\FileStorage\Request001\Branch1\...\doc.pdf	\\xxx.file.core.windows.net\fileshare\Request001\Branch1\SealType1\doc.pdf

6.2 Web.config / appsettings.json 配置

6.3 统一文件服务接口

```
public interface IFileStorageService { /// ██████████ string GetFullPath(string relativePath); ///
██████████ bool FileExists(string relativePath); /// ██████ byte[] ReadFile(string relativePath); /// █████
void WriteFile(string relativePath, byte[] content); /// █████ void CreateDirectory(string relativePath);
/// █████ void MoveFile(string sourcePath, string destPath); /// █████ void DeleteFile(string
relativePath); /// █████████ IEnumerale GetFiles(string relativePath, string searchPattern); }
```


7. 完整迁移架构设计

7.1 迁移后的完整架构

组件	技术选型	说明
Web 应用	ASP.NET MVC	保持不变，集成 AzureFileStorageService
数据库	SQL Server / Azure SQL	FilePath 字段保持相对路径，不变
文件存储	Azure File Storage	替换本地文件系统
SMB 挂载	PowerShell 脚本	Windows Service 服务器挂载 Azure File Share
Windows Service	.NET Windows Service	无需修改代码，直接访问挂载路径

7.2 SMB 挂载方案

将 Azure File Share 挂载为 Windows 网络驱动器，Windows Service 可以像访问本地磁盘一样 访问 Azure 文件。迁移后应用程序和 Service 代码零改动。

```
# PowerShell: ■■■ Azure File Share ■■■■■■ # 1. ■■■■■■■■ $StorageAccountName = "mystorageaccount"
$StorageAccountKey = (Get-AzStorageAccountKey -Name $StorageAccountName -ResourceGroupName "myRG")[0].Value
# 2. ■■■■ $credential = New-Object System.Management.Automation.PSCredential -ArgumentList
"Azure\$StorageAccountName", (ConvertTo-SecureString $StorageAccountKey -AsPlainText -Force) # 3.
■■■■■■■■ (Z:) New-PSDrive -Name Z -PSProvider FileSystem -Root
"\\mystorageaccount.file.core.windows.net\fileshare" -Credential $credential -Persist -RememberMe # 4.
■■■■ Get-ChildItem Z:/ # 5. ■■■■■■■■■■■■■■■■■■■■■■ # ■■■■ Azure Automation Runbook ■ VM ■■■■■
```

7.3 迁移后的文件访问路径

场景	迁移前	迁移后
用户上传	Controller 保存到 D:\FileStorage\[RequestID]\	Controller 保存到 Z:\[RequestID]\ (Z: 是挂载的 Azure File Share)
Windows Service 扫描	D:\FileStorage\[RequestID]\	Z:\[RequestID]\ (路径完全不变)
文件重分类	File.Move 到 D:\FileStorage\[RequestID]\[Branch]\	File.Move 到 Z:\[RequestID]\[Branch]\
数据库存储	\Request001\doc.pdf	\Request001\doc.pdf (完全不变)
URL 访问	http://app/Download?path=... \doc.pdf	http://app/Download?path=... \doc.pdf (不变, 需映射虚拟目录)

8. ASP.NET 代码改造方案

8.1 新增文件服务实现

```
using Microsoft.WindowsAzure.Storage.File; using System; using System.IO; using System.Configuration;
public class AzureFileStorageService : IFileStorageService { private readonly string _rootPath; public
AzureFileStorageService() { // ■■■■■ Azure File Share ■■■ _rootPath =
ConfigurationManager.AppSettings["FileStorage:RootPath"]; } public string GetFullPath(string relativePath)
{ // ■■■■■ relativePath = relativePath.Replace("/", "\\"); return Path.Combine(_rootPath,
relativePath); } public bool FileExists(string relativePath) { return
File.Exists(GetFullPath(relativePath)); } public byte[] ReadFile(string relativePath) { return
File.ReadAllBytes(GetFullPath(relativePath)); } public void WriteFile(string relativePath, byte[] content)
{ var fullPath = GetFullPath(relativePath); var directory = Path.GetDirectoryName(fullPath); if
(!Directory.Exists(directory)) Directory.CreateDirectory(directory); File.WriteAllBytes(fullPath, content);
} public void CreateDirectory(string relativePath) { Directory.CreateDirectory(GetFullPath(relativePath));
} public void MoveFile(string sourcePath, string destPath) { File.Move(GetFullPath(sourcePath),
GetFullPath(destPath)); } public void DeleteFile(string relativePath) {
File.Delete(GetFullPath(relativePath)); } public IEnumerable GetFiles(string relativePath, string
searchPattern) { return Directory.GetFiles(GetFullPath(relativePath), searchPattern); } }
```

8.2 Controller 改造示例

[illegible]

9. 迁移实施步骤

9.1 实施计划

阶段	任务	说明
1. 准备	创建 Azure Storage Account	选择标准 LRS，启用 File Share
1. 准备	部署文件服务代码	新增 IFileStorageService 和 AzureFileStorageService
1. 准备	备份数据库	完整备份 SQL Server 数据库
2. 测试	SMB 挂载测试	在测试环境挂载 Azure File Share
2. 测试	文件迁移测试	使用 AzCopy 迁移测试数据
2. 测试	功能回归测试	验证上传、下载、Service 重分类功能
3. 灰度	单台服务器切换	一台 Web Server + 一台 Service 切换到 Azure
3. 灰度	监控验证	观察日志、错误率、性能指标
4. 全量	全部切换	所有服务器切换到 Azure File Share
4. 全量	旧数据清理	可选：清理本地旧文件

9.2 文件迁移命令

```
# ■■ AzCopy ■■■■■■■■ Azure File Share # 1. ■■ AzCopy #  
https://docs.microsoft.com/zh-cn/azure/storage/common/storage-use-azcopy-v10 # 2. ■■ Azure azcopy login #  
3. ■■■■■■■■ File Share azcopy copy "D:\FileStorage\*" #  
"https://mystorageaccount.file.core.windows.net/fileshare" --recursive=true # 4. ■■■■■■■■ #
```

9.3 回滾方案

- 方案A: 卸载 SMB 挂载, Windows Service 自动回退到本地磁盘 (D:\)
- 方案B: 修改 Web.config 的 RootPath 配置, 切换回本地路径
- 方案C: 数据库不变, 应用程序通过 IFileStorageService 切换实现

10. 总结与建议

10.1 核心结论

问题	结论	理由
存储类型选哪个?	Azure File Storage	完美支持 SMB、层级目录，与现有架构最高兼容
数据库路径要改吗?	不需要改	保持 FilePath 相对路径不变，通过配置映射到 Azure
Windows Service 要改吗?	不需要改	通过 SMB 挂载，Service 代码零改动
ASP.NET MVC 要改吗?	小幅改动	新增文件服务接口，通过 DI 切换本地/Azure 实现

10.2 迁移收益

- + 高可用：Azure File Share 默认 3 副本冗余，消除单点故障
- + 弹性扩展：按需扩展存储容量，无需担心磁盘满
- + 低成本运维：无需管理文件服务器硬件和备份
- + 全球化访问：支持跨区域访问和 CDN 加速
- + 安全合规：支持 Azure AD 认证、RBAC、加密传输

10.3 后续优化建议

- 短期：完成 SMB 挂载迁移，验证功能正常
- 中期：使用 Azure File Sync 实现本地缓存 + 云端统一的混合方案
- 长期：考虑迁移到 Blob Storage + CDN，通过应用层统一管理文件访问

文档说明：本分析基于 ASP.NET Framework MVC + Windows Service 架构，针对 PDF/Office 文档存储和定期重分类场景，推荐使用 Azure File Storage 并保持数据库路径字段不变，通过 SMB 挂载实现平滑迁移。